

Author

Author Name: Labangi Mahata

Roll Number: DS23F2000474

Email: 23f2000474@ds.study.iitm.ac.in

About Me: I am a student pursuing the BS in Data Science and Applications at IIT Madras, with a strong interest in data science, machine learning, and data-driven technologies. I am passionate about exploring how data can be transformed into meaningful insights and intelligent solutions. I enjoy building meaningful applications that combine learning, analytics, and user experience.

AI/LLM usage

I used ChatGPT (GPT-5.1) minimally to assist with debugging specific issues and clarifying error messages during development. The overall extent of AI/LLM usage was limited to approximately 3-5% and was confined to troubleshooting support only. All final system design, core implementation logic, feature development, SQLAlchemy model, and testing were completed independently and manually.

Description

The solution is implemented as a **role-based web application** where Admin, Company, and Student users interact through separate dashboards. A **Flask backend with an SQLite database** manages entities like companies, students, placement drives, and applications with defined relationships. The system enables job posting, student applications, admin approvals, and application status tracking.
. The overall extent of AI/LLM usage was limited to approximately 3-5% and was confined to troubleshooting support only.

Technologies used

Technology/Library	Purpose
• Flask	Core backend web framework
• SQLAlchemy	Object Relational Mapper for SQLite database
• Jinja2	Template engine for rendering dynamic HTML pages
• Bootstrap 5	Frontend styling and responsive design
• SQLite	Lightweight local database for storing user data

DB Schema Design

1. User

Stores authentication and authorization details for all system users including **admin, student, and company accounts**.

Columns:

(user_id, username, password, role, status)

2. Student

Stores academic and personal information of students linked to a user account.

Columns:

(student_id, user_id, full_name, course, branch, graduation_year, cgpa, contact, skills, project, cv_file)

3. Company

Stores company details and approval status for companies participating in placement drives.

Columns:

(company_id, user_id, name, hr_contact, website, approval_status)

4. PlacementDrive

Stores job postings or placement drives created by companies.

Columns:

(drive_id, company_id, job_title, job_description, eligibility, salary, location, deadline, status, created_at)

5. Application

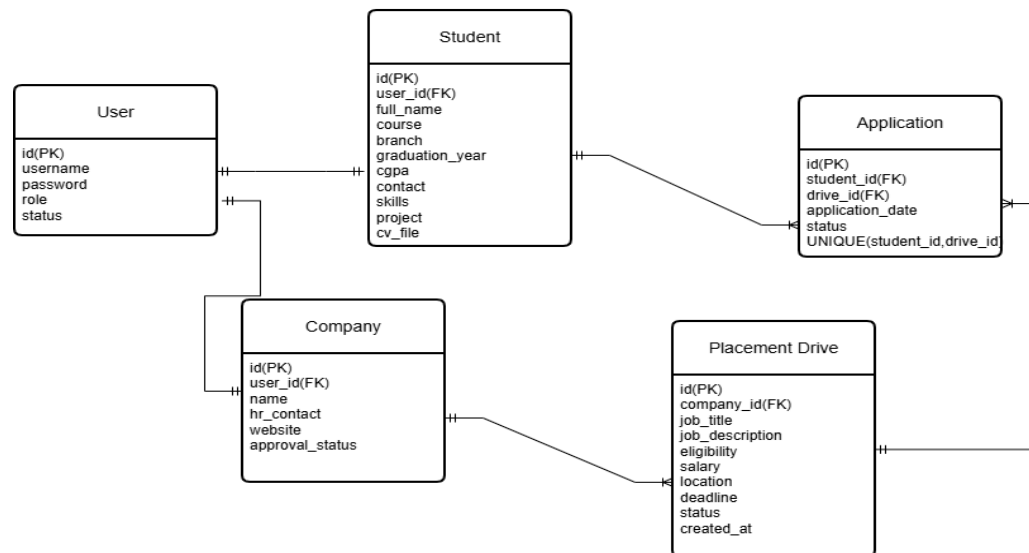
Stores job applications submitted by students for placement drives.

Constraint:

- **Unique(student_id, drive_id)** ensures a student cannot apply for the same placement drive more than once.

Relationships

- **One-to-One → User → Student**
Each student profile is associated with one user account.
- **One-to-One → User → Company**
Each company profile is linked to one user account.
- **One-to-Many → Company → PlacementDrive**
A company can create multiple placement drives.
- **One-to-Many → Student → Application**
A student can submit multiple job applications.
- **One-to-Many → PlacementDrive → Application**
A placement drive can receive multiple student applications.



API Design

No use of API Resource Endpoints.

Architecture and Features

Architecture Overview:

- **app.py** – main Flask application entry point with controllers in the routes defined
- **templates** – Jinja2 HTML templates

Features

The Placement Portal Application implements a role-based system with Admin, Company, and Student users. The system supports user authentication, company approval by the admin, and role-specific dashboards for managing placement activities. Companies can create and manage placement drives, review student applications, and update application statuses, while students can create profiles, upload resumes, view approved job postings, and apply for positions. The system also maintains application history, prevents duplicate applications, and allows tracking of application status throughout the recruitment process.

Video

[Demo Video](#) (Bootstrap for backend in the last part of video was a slip of tongue and meant to convey in the frontend for the purpose stated like I have mentioned in the earlier parts of the video)